

BCanteen

SOFTWARE DEVELOPMENT LIFE CYCLE

1. Project Overview

BCanteen is a digital foodcourt ordering platform developed as a Software Engineering final project. The system enables customers to browse foodcourts, explore tenant stalls, view menus, add items to cart, and complete orders without requiring registration or personal data. Upon successful checkout, the system automatically generates a unique order code (e.g., BCA-0001) which the customer presents at the tenant counter to collect their food. The platform addresses a common pain point in workplace foodcourts: long queues caused by on-site ordering. By digitizing the ordering process, BCanteen allows customers to browse and order in advance, reducing congestion and improving the overall dining experience.

Key Business Rules

- Customers do NOT need to register or log in
- No personal information (name, email, phone) is collected
- Hierarchy: Foodcourt → Tenants → Menus
- Cart is locked to a single tenant per transaction
- Order codes are auto-generated in the format BCA-XXXX
- Order status lifecycle: Pending → Preparing → Ready → Completed

Technology Stack

Layer	Technology	Purpose
Frontend	Vue.js 3 + Tailwind CSS	Reactive UI, component-based architecture
State Management	Pinia	Cart state & reactive store
Routing	Vue Router	SPA client-side navigation
HTTP Client	Axios	REST API communication
Backend	Node.js + Express.js	REST API server, business logic
Database	MySQL 8.0	Relational data persistence
Architecture	REST API (MVC pattern)	Separation of concerns

2. Software Development Life Cycle (SDLC) Model

2.1 Model Selection: Iterative & Incremental

BCanteen was developed using the Iterative & Incremental Development Model. This model was selected because it allows the development team to deliver functional software in successive increments, with each iteration refining and expanding upon the previous one. Unlike the Waterfall model, which requires all requirements to be finalized upfront, the Iterative & Incremental model accommodates evolving design decisions, particularly important for a project where the UI/UX design (Figma) was used as the primary reference and required continuous interpretation and refinement during implementation.

Why Not Other Models?

Model	Reason for Consideration	Reason Not Chosen
Waterfall	Simple, well-documented, easy to understand for academic projects	Inflexible; all requirements must be known upfront. Design changes mid-project cause expensive rework.
Agile / Scrum	Highly flexible, sprint-based, widely used in industry	Requires active stakeholder collaboration and daily standups. Overhead is disproportionate for a single-team academic project.
Iterative & Incremental	Delivers working increments; allows early feedback; manageable for small teams	CHOSEN, best balance between structure and flexibility for this project scope.
Prototype	Good for UI-heavy applications with unclear requirements	Requirements were reasonably well-defined via Figma designs; full prototyping phase unnecessary.

2.2 Core Concepts of the Iterative & Incremental Model

The Iterative & Incremental Model combines two complementary development strategies. Iteration refers to the repeated execution of a defined process cycle, plan, design, implement, and test, where each cycle produces a progressively more complete version of the software. Incremental development means that the system is built and delivered in functional pieces (increments), with each increment adding new capability to the existing working system.

In the context of BCanteen, each phase of development represented a distinct increment:

- Increment 1 Backend Core: Database schema, seed data, and all REST API endpoints were delivered as a fully testable, standalone increment.
- Increment 2 Frontend Core: Vue 3 pages (Landing, Home, Foodcourt, Tenant Menu) were implemented as a working UI increment connected to the API.
- Increment 3 Integration & Checkout: Cart state management, checkout flow, order code generation, and the order success page were delivered as the final functional increment.

Each increment was tested independently before the next was started, ensuring that defects were isolated and resolved early rather than discovered at the end of the project lifecycle.

2.3 Development Phases

The BCanteen project was structured into six sequential phases over a period of approximately seven weeks (one and a half months). Each phase corresponds to a development iteration with defined deliverables.

Phase 1 Week 1-2	Requirements & Planning Defined functional and non-functional requirements. Analyzed Figma screens to identify all pages, components, and user flows. Finalized technology stack. Identified core business rules: no user registration, order code generation format, cart single-tenant constraint, and order status lifecycle.	✓ Completed
Phase 2 Week 2-3	System Design Designed the complete system architecture including folder structure, routing table, component hierarchy, API contracts, and database ERD. UI/UX designs were analyzed from Figma. The three-layer entity model (Foodcourt → Tenants → Menus) was established and propagated through all design artifacts.	✓ Completed
Phase 3 Week 3-4	Iteration 1: Backend Core Implemented the MySQL schema with all five tables, constraints, indexes, and ENUM status values. Created 30+ seed records for demo-readiness. Built the Express.js application with all controllers, services,	✓ Completed

	routes, middleware, and the automatic order code generator (BCA-XXXX format).	
Phase 4 Week 4-5	Iteration 2: Frontend Core Built the Vue 3 Single Page Application with Vite, configured Tailwind CSS, and implemented Vue Router. Developed all primary pages (Landing, Home, Foodcourt List, Foodcourt Detail, Tenant Menu) and reusable components following the Figma design system precisely.	✓ Completed
Phase 5 Week 5-6	Iteration 3: Integration Connected the frontend to the backend via Axios service layer. Implemented the Pinia cart store with single-tenant enforcement. Built the Checkout page, Place Order flow, and Order Success page. Validated the complete end-to-end customer journey from homepage to order code display.	✓ Completed
Phase 6 Week 6-7	Testing & Finalization Conducted end-to-end flow testing across all pages. Resolved UI inconsistencies identified during integration. Finalized README, setup guide, API documentation, and demo seed data. Prepared the project for submission and live demonstration.	✓ Completed

3. Product Backlog

The product backlog defines the complete set of features, user stories, and implementation tasks for BCanteen MVP. Items are prioritized using the MoSCoW method (High / Medium / Low) and tracked through the development lifecycle.

Feature	User Story	Task	Priority	Status
---------	------------	------	----------	--------

Home Page	Customer browses featured foodcourts and categories on the landing screen	Design responsive home layout with search bar, category chips, and featured cards	High	Done
		Implement GET /api/foodcourts and bind data to Vue components	High	Done
Foodcourt List	Customer views all available foodcourts with ratings and distance	Build FoodcourtListPage.vue with card grid and search filter	High	Done
Foodcourt Detail	Customer views tenants inside a foodcourt and filters by cuisine	Build FoodcourtDetailPage.vue with tenant cards and category filter chips	High	Done
Tenant Menu	Customer browses menus grouped by category inside a tenant stall	Build TenantMenuPage.vue with category tabs and MenuItemCard components	High	Done
		Implement GET /api/tenants/:id/menus and category filter logic	High	Done
Add to Cart	Customer adds menu items to cart from the tenant menu page	Implement Pinia cartStore with addItem, removeItem, updateQuantity actions	High	Done
		Enforce single-tenant cart rule with conflict prompt dialog	High	Done
Cart Review	Customer reviews cart items, adjusts quantities, and sees total price	Build CartPage.vue with quantity controls and real-time total calculation	High	Done

Checkout	Customer reviews order summary and confirms purchase	Build CheckoutPage.vue with order summary, special notes, and payment UI	High	Done
		Display QRIS and M-Bandini as UI-only payment method selectors	High	Done
Order Code	System generates unique order code after checkout	Implement orderCodeGenerator.js with BCA-XXXX sequential format	High	Done
		Implement POST /api/orders with atomic insert of order + order_items	High	Done
Order Success	Customer sees generated order code and order summary	Build OrderSuccessPage.vue displaying order code, items, total, and status	High	Done
Search	Customer searches for foodcourts or menu items by keyword	Implement search query param on GET /api/foodcourts and bind to SearchBar component	Medium	Done
Seed Data	App is demo-ready immediately after database setup	Create seed SQL: 3 foodcourts, 6 tenants, 30 menu items with realistic Indonesian data	Medium	Done
Error Handling	App handles API errors gracefully without crashing	Implement global errorHandler middleware and Axios interceptor for error states	Medium	Done
Management Side	Admin manages tenants, menus, and order statuses	Deferred to v2.0, outside MVP scope	Low	Deferred

4. Project Timeline: Gantt Chart

The following Gantt chart illustrates the planned development schedule across seven weeks (~1.5 months). Orange cells indicate active development periods for each task.

Activity / Task	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7
PHASE 1: Requirements & Planning							
Define project scope & objectives	■						
Stakeholder analysis	■						
Functional requirements	■	■					
Non-functional requirements		■					
Technology stack finalization		■					
PHASE 2: System Design							
Database design & ERD		■	■				
API design & documentation		■	■				
UI/UX Figma wireframing		■	■				
Project folder structure			■				
PHASE 3: Iteration 1 (Backend Core)							
MySQL schema implementation			■				
Seed data & demo setup			■				
Express app & middleware			■	■			

Foodcourt & tenant APIs							
Menu APIs							
Order APIs + code generator							
PHASE 4: Iteration 2 (Frontend Core)							
Vue 3 project setup & routing							
Landing & Home page							
Foodcourt list & detail pages							
Tenant menu page							
Cart store (Pinia)							
PHASE 5: Iteration 3 (Integration)							
Axios API service layer							
Checkout & order flow							
Order success page							
Frontend ↔ Backend integration							
PHASE 6: Testing & Finalization							
End-to-end flow testing							
Bug fixing & UI polish							
README & setup guide							
Demo preparation							

Legend: Active Development Period Inactive

5. System Design Overview

5.1 Entity Relationship Design

The database follows a normalized relational design with five core tables. The three-tier entity hierarchy (Foodcourt → Tenants → Menus) mirrors the real-world structure of a foodcourt facility.

Table	Relation	Key Columns
foodcourts	Root entity	id, name, location, rating, distance_km, updated_at
tenants	→ foodcourts	id, foodcourt_id (FK), name, cuisine_type, is_open, prep_time_min
menus	→ tenants	id, tenant_id (FK), name, price (DECIMAL), category, is_available
orders	→ tenants	id, order_code (UNIQUE), tenant_id (FK), total_price, status (ENUM), notes
order_items	→ orders, menus	id, order_id (FK), menu_id (FK), quantity, subtotal

5.2 API Design

Method	Endpoint	Description
GET	/api/foodcourts	List all foodcourts with optional search query
GET	/api/foodcourts/:id	Get single foodcourt details
GET	/api/foodcourts/:id/tenants	Get all tenants in a foodcourt
GET	/api/tenants/:id	Get single tenant details
GET	/api/tenants/:id/menus	Get all menus for a tenant with optional category filter
GET	/api/menus/:id	Get single menu item details
POST	/api/orders	Create order, generate order code, return BCA-XXXX
GET	/api/orders/:orderCode	Retrieve order by code for order success page

5.3 Order Code Generation Algorithm

The order code generation follows a simple, predictable sequential format:

- Query the maximum existing order code from the orders table
- Extract the numeric suffix and increment by 1
- Zero-pad to 4 digits (e.g., 1 → 0001)
- Prepend the prefix BCA- to form the final code
- The operation is performed atomically within the order creation transaction

Example progression: BCA-0001 → BCA-0002 → BCA-0003 → ... → BCA-9999

Future Enhancements (v2.0):

- Management dashboard for tenant owners to manage menus and update order status
- Real-time order status tracking via WebSocket
- Customer order history via session-based token
- Push notifications when order status changes to Ready Analytics dashboard for foodcourt management